

Exercice type examen — walk() + copie partielle

Sujet

Écrire un script Python qui :

1. Prend en argument :
 - un dossier source
 - un nom de fichier à rechercher
 - un nombre de lignes à copier
2. Parcourt récursivement le dossier avec `Path.walk()`
3. Trouve le fichier correspondant
4. Copie seulement les **N premières lignes** de ce fichier
5. Écrit ces lignes dans un fichier de destination nommé :

```
copie_<nom_du_fichier>
```

6. Le fichier de destination doit être créé dans le dossier courant

Solution

```
from pathlib import Path
import argparse

parser = argparse.ArgumentParser(description="Copie partielle d'un fichier")
parser.add_argument("dossier", type=str, help="Dossier à parcourir")
parser.add_argument("nom_fichier", type=str, help="Nom du fichier à trouver")
parser.add_argument("nb_lignes", type=int, help="Nombre de lignes à copier")

args = parser.parse_args()

base = Path(args.dossier)

fichier_trouve = None

# Parcours avec walk()
for racine, dossiers, fichiers in base.walk():
    for nom in fichiers:
        if nom == args.nom_fichier:
            fichier_trouve = racine / nom
            break
    if fichier_trouve:
        break
```

```
if not fichier_trouve:
    print("Fichier non trouvé")
    exit()

# Lecture partielle
lignes_a_copier = []

with fichier_trouve.open("r") as f:
    for i, ligne in enumerate(f):
        if i >= args.nb_lignes:
            break
        lignes_a_copier.append(ligne)

# Écriture dans nouveau fichier
destination = Path(f"copie_{args.nom_fichier}")

with destination.open("w") as f:
    f.writelines(lignes_a_copier)

print("Copie terminée :", destination)
```

Variante probable (très important)

Ton prof peut changer légèrement :

Variante 1 — copier une plage de lignes

Sujet

Copier les lignes entre `ligne_debut` et `ligne_fin`

Logique à connaître

```
for i, ligne in enumerate(f):
    if i >= debut and i <= fin:
        lignes.append(ligne)
```

Variante 2 — copier seulement les lignes contenant un mot

Sujet

Copier uniquement les lignes contenant un mot donné

```
if mot in ligne:
    lignes.append(ligne)
```

Variante 3 — copier depuis un mot jusqu'à la fin

```
copie = False

for ligne in f:
    if mot in ligne:
        copie = True
    if copie:
        lignes.append(ligne)
```

Pièges que ton prof peut mettre

Très important :

- oublier de faire `racine / nom`
- confondre `Path` et `string`
- oublier `break` (continue à chercher inutilement)
- écrire avec `write()` au lieu de `writelines()`
- oublier `enumerate()`
- ne pas arrêter la lecture (`break`)

Mini exercice bonus (rapide)

Sujet

Parcourir un dossier avec `walk()` et afficher :

- nom du fichier
- taille en KO

Solution

```
from pathlib import Path

base = Path(".")

for racine, dossiers, fichiers in base.walk():
    for nom in fichiers:
        chemin = racine / nom
        taille = chemin.stat().st_size / 1024
        print(nom, "-", taille, "KO")
```

Exercice type examen — walk() / rglob() + copie partielle

Sujet

Créer un script Python qui :

1. Prend en argument :
 - un dossier à analyser
 - une extension de fichier (ex: `.txt`)
 - un mot-clé
2. Parcourt récursivement le dossier (avec `walk()` ou `rglob()`)
3. Pour chaque fichier correspondant à l'extension :
 - lit le contenu
 - extrait uniquement les lignes contenant le mot-clé
4. Copie ces lignes dans un fichier de sortie nommé :

`resultat.txt`

5. Le fichier de sortie doit contenir toutes les lignes trouvées dans tous les fichiers

Solution (version avec walk)

```
from pathlib import Path
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("dossier", type=str)
parser.add_argument("extension", type=str)
parser.add_argument("mot", type=str)

args = parser.parse_args()

base = Path(args.dossier)
resultat = []

for racine, dossiers, fichiers in base.walk():
    for nom in fichiers:
        if nom.endswith(args.extension):
            chemin = racine / nom

            with chemin.open("r") as f:
                for ligne in f:
                    if args.mot in ligne:
```

```
        resultat.append(ligne)

# écriture finale
with open("resultat.txt", "w") as f:
    f.writelines(resultat)

print("Extraction terminée")
```

Solution (version avec rglob)

```
from pathlib import Path
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("dossier", type=str)
parser.add_argument("extension", type=str)
parser.add_argument("mot", type=str)

args = parser.parse_args()

base = Path(args.dossier)
resultat = []

for fichier in base.rglob(f"*{args.extension}"):
    if fichier.is_file():
        with fichier.open("r") as f:
            for ligne in f:
                if args.mot in ligne:
                    resultat.append(ligne)

with open("resultat.txt", "w") as f:
    f.writelines(resultat)

print("Extraction terminée")
```

Variante très probable (à maîtriser absolument)

Copier seulement N lignes de chaque fichier

```
nb_lignes = 3

with fichier.open("r") as f:
```

```
for i, ligne in enumerate(f):
    if i >= nb_lignes:
        break
    resultat.append(ligne)
```

Copier uniquement une plage de lignes

```
debut = 2
fin = 5

for i, ligne in enumerate(f):
    if i >= debut and i <= fin:
        resultat.append(ligne)
```

Copier à partir d'un mot

```
copie = False

for ligne in f:
    if args.mot in ligne:
        copie = True
    if copie:
        resultat.append(ligne)
```

Ce que tu dois absolument maîtriser

- `Path.walk()` structure :

```
for racine, dossiers, fichiers in base.walk():
```

- construction du chemin :

```
chemin = racine / nom
```

- lecture fichier :

```
with chemin.open("r") as f:
```

- écriture multiple :

```
f.writelines(lignes)
```

- `enumerate()` pour lignes
 - conditions sur string (`in` , `endswith`)
-